Generation Probabilities Are Not Enough: What Should AI Code Assistants Highlight?

A NeurIPS 2022 study from Stanford and Microsoft Research finds that highlighting the tokens a person is likely to edit beats highlighting the tokens a model happened to be unsure about.

AI pair programmers like GitHub Copilot, Amazon CodeWhisperer, and DeepMind's AlphaCode now suggest code completions right inside the editor. They are useful, but they are imperfect, and a wrong suggestion can quietly plant a bug or a security vulnerability if a human waves it through. To override a bad suggestion, a programmer first has to notice it, and that is harder than it sounds: even experts are prone to automation bias, the tendency to over-trust a confident machine.

A natural fix, borrowed from the inline spell-checker, is to highlight the parts of a suggestion that most need a second look. But a code completion is not one decision; it is hundreds of small ones, one per token. Which tokens should you highlight? The obvious answer is the ones the model itself was least sure about, its lowest generation probabilities, on the theory that low-probability text is unconventional and therefore suspect. This strategy has been proposed before and even ships in OpenAI's Playground. This paper asks a sharper question: are the model's own generation probabilities actually the right thing to be highlighting in the first place?

In a preregistered, mixed-methods study with 30 programmers, the authors put generation-probability highlighting head to head against a new "edit model" that predicts which tokens a person is likely to change. The reframing is the whole point: instead of asking where the model was uncertain, ask where a human is likely to intervene. Across task time, edit precision, and user preference, the edit model wins, and generation-probability highlighting turns out to be the worst of the options tested, worse even than showing no highlights at all.

Paper: https://arxiv.org/abs/2302.07248

Preregistration: https://osf.io/tymah

The obvious signal, and why it might be the wrong one

Highlighting low-probability tokens has an appealing logic. A language model assigns a probability to each token it emits, and the tokens it hesitated over, the ones it assigned low probability, are plausibly the ones most likely to be wrong. Underline them, and the programmer's eye is pulled toward the risky spots, in much the same way a spell-checker draws a squiggly line beneath a word it suspects you have misspelled.

The catch is that model uncertainty and human need are not the same thing. A model can be perfectly confident about a token that a person will still want to change to fit their own style, variable names, or edge cases, and it can waver over a token that is in fact correct and will be left untouched. What a programmer actually needs highlighted is not where the model was

unsure, but where they are likely to reach in and edit. That is the hypothesis this study sets out to test.

One model, three faces

To isolate the effect of the highlighting signal, the authors held everything else fixed. All three tools in the study were driven by the same underlying model, OpenAI's Davinci-002 Codex, and produced identical completions; they differed only in how those completions were presented. The Prediction-only tool showed the raw completion with no highlights. The Generation-probability tool highlighted the tokens Codex was least confident about, at a probability threshold of 71%. The Edit-model tool highlighted the tokens most likely to be edited, at a threshold of 66%, according to a learned model of user edits.

That edit model was trained during a preliminary phase on 9 coders who edited Codex output until each task was solved correctly, giving the model examples of which tokens humans actually touch. Crucially, the two thresholds were tuned so that every condition displayed the same total number of highlights across all tasks. That design choice matters: it means the comparison is about which tokens get highlighted, not how many, so any difference in behavior comes down to the quality of the signal.

The 30 participants were experienced Python programmers at a large US technology company, each paid $50 for a roughly one-hour session. Every person solved three coding problems drawn from LeetCode's "easy" tier, with a 10-minute cap per task, and could run their code against a set of provided unit tests to debug. Task order and the assignment of tools to tasks were randomized to keep the comparison fair.

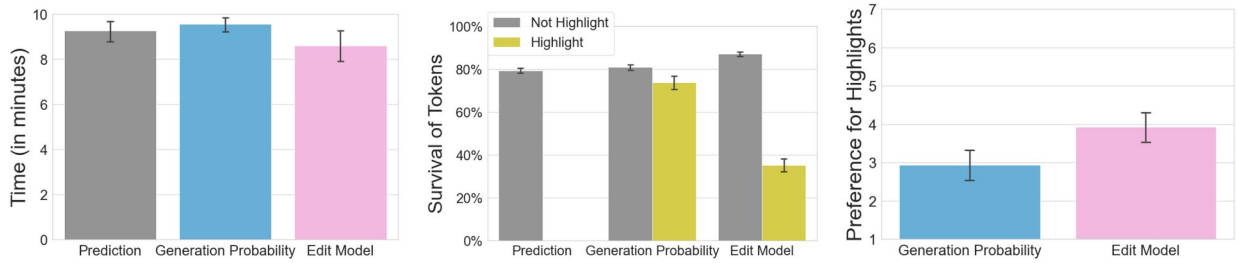

Figure 1. (a) Time spent on each task (10-minute cap). (b) Survival of tokens in each condition, split by whether the token was highlighted. (c) Self-reported preference for the highlights. The edit model is fastest, its highlights are the least likely to survive (they closely predict real edits), and it is the preferred tool.

Highlighting the right tokens made people faster

The headline result is about speed (Figure 1a). Participants finished fastest with edit-model highlighting, averaging 8.59 minutes per task, and slowest with generation-probability highlighting, at 9.61 minutes, with the no-highlight Prediction-only condition sitting in between at 9.27 minutes. The gap between the two highlighting strategies is highly significant ($p = 0.003$), while the edit model's edge over showing no highlights at all is a promising but weaker trend ($p = 0.06$), which points to the specific choice of signal, rather than the mere presence of highlights, as what drives the benefit.

The uncomfortable finding is at the other end of the ranking: generation-probability highlighting was not just beaten by the edit model, it was the slowest of the three, slower even than Prediction-only. Highlighting the wrong tokens did not merely fail to help; it appears to have actively slowed people down, presumably by drawing attention to code that did not need it. What you highlight matters more than whether you highlight at all.

Table 1. Edit-model highlighting versus generation-probability highlighting on the study's three headline measures. Survival is the fraction of highlighted tokens left unchanged; a lower value means the highlights better predict real edits.

Measure	Generation Probability	Edit Model	p
Task time (minutes)	9.61	8.59	0.003
Highlighted-token survival	0.74	0.35	< 0.0001
Preference (7-point scale)	2.88	3.94	0.001

Why it works: edit-likelihood tracks what people actually change

To see why the edit model helps, the authors tracked which tokens survived, meaning they were left unchanged by the participant (Figure 1b). A good highlight should mark tokens that get edited and leave alone the tokens that stay. The edit model does exactly that. The tokens it left un-highlighted survived 0.87 of the time, more than under generation probability (0.81) or Prediction-only (0.79), while the tokens it did highlight survived only 0.35 of the time. Generation probability was far muddier: its highlighted tokens still survived 0.74 of the time, barely different from tokens it left alone. All of these differences are extremely significant ($p < 0.0001$).

In plain terms, the edit model's highlights closely predict what people will actually change, while generation-probability highlights are only weakly aligned with real edits. Participants felt the difference too (Figure 1c): on a 7-point scale combining how helpful, worth-paying-for, and non-distracting they found the highlights, the edit model scored 3.94 against 2.88 for generation probability, once again a significant gap at $p = 0.001$.

What it means, and what is still open

The takeaway is simple but pointed: to help someone review AI-generated code, highlight where they are likely to edit, not where the model happened to be unsure. That single change made programmers faster, more surgical in their edits, and left them clearly preferring the tool. Generation probabilities, the signal that is easiest to read off a language model and already shipping in real interfaces, are not enough.

The authors are candid about the limits. Their edit model is closed-world: it was trained on the very AI generations it was later evaluated on, a best-case, near-perfectly-calibrated setting. Whether an open-world, general-purpose edit model can be learned, and whether it would move behavior the same way, is still to be shown. It also remains to be demonstrated that these gains in speed, precision, and preference translate into the outcomes that ultimately matter, such as more accurate human oversight and less automation bias. The encouraging note is that products like GitHub Copilot already log edits to their suggestions as a performance metric, and that product-scale stream of real edits is exactly the kind of data an open-world edit model would need to get off the ground.